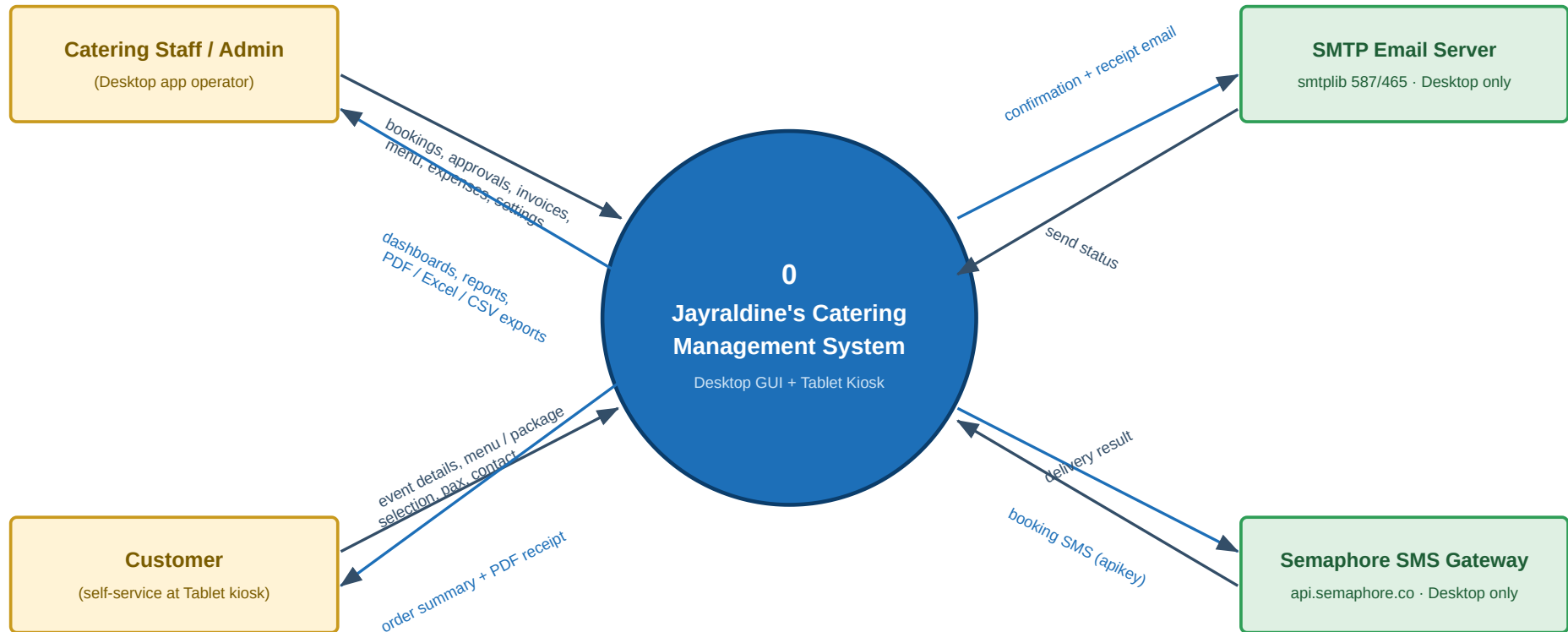


Jayraldine's Catering Management System — Data Flow Diagram

Page 1 · Context Diagram (Level 0)

Two client apps (PySide6 Desktop + Tablet PWA kiosk) over one LAN sync hub · verified against source · 2026-10-07



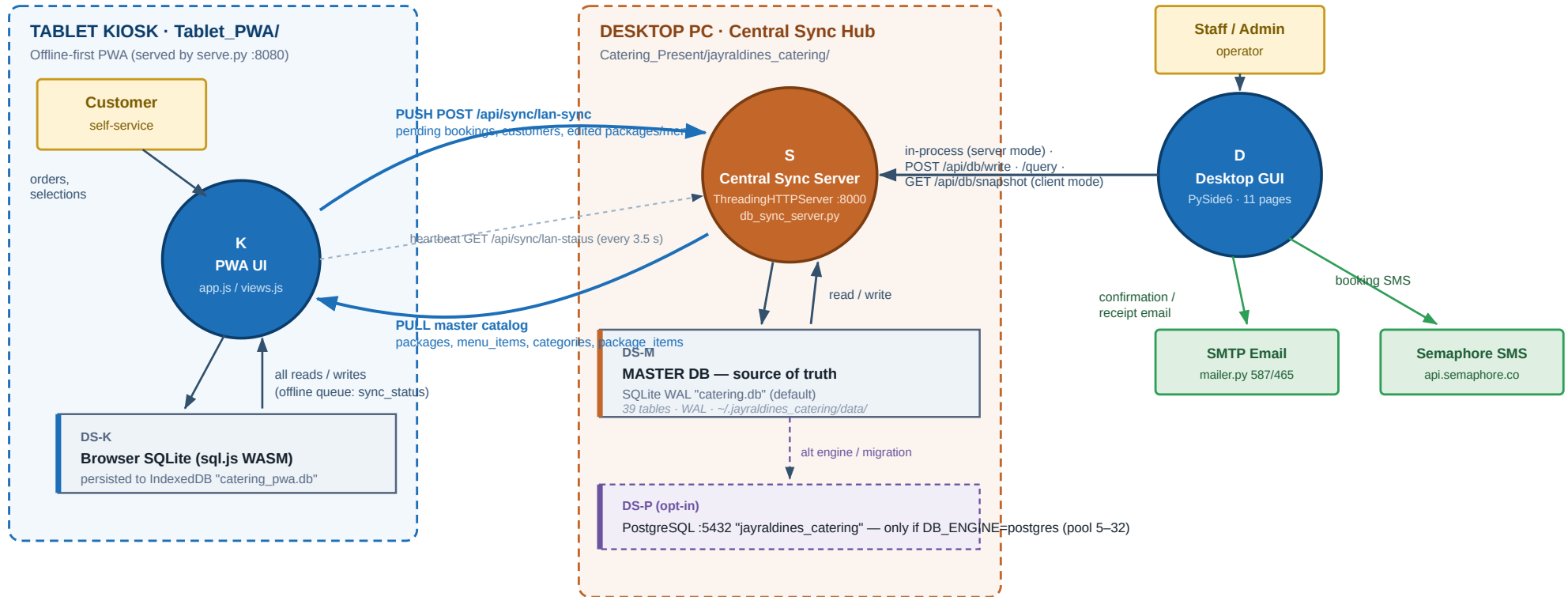
Internal user (entity) External service System (process 0) Data flow

Staff drive the PySide6 desktop app; customers interact through the Tablet PWA kiosk. Email (SMTP) and SMS (Semaphore) are triggered only from the desktop side — the kiosk has no messaging integration.

Data Flow Diagram — Page 2 · Deployment & LAN Sync Topology

How the Tablet kiosk, the Desktop app, the Central Sync Server and the data stores actually exchange data (code-verified)

All sync = HTTP-JSON on port 8000 · master store defaults to SQLite (WAL); PostgreSQL:5432 only on the FastAPI path

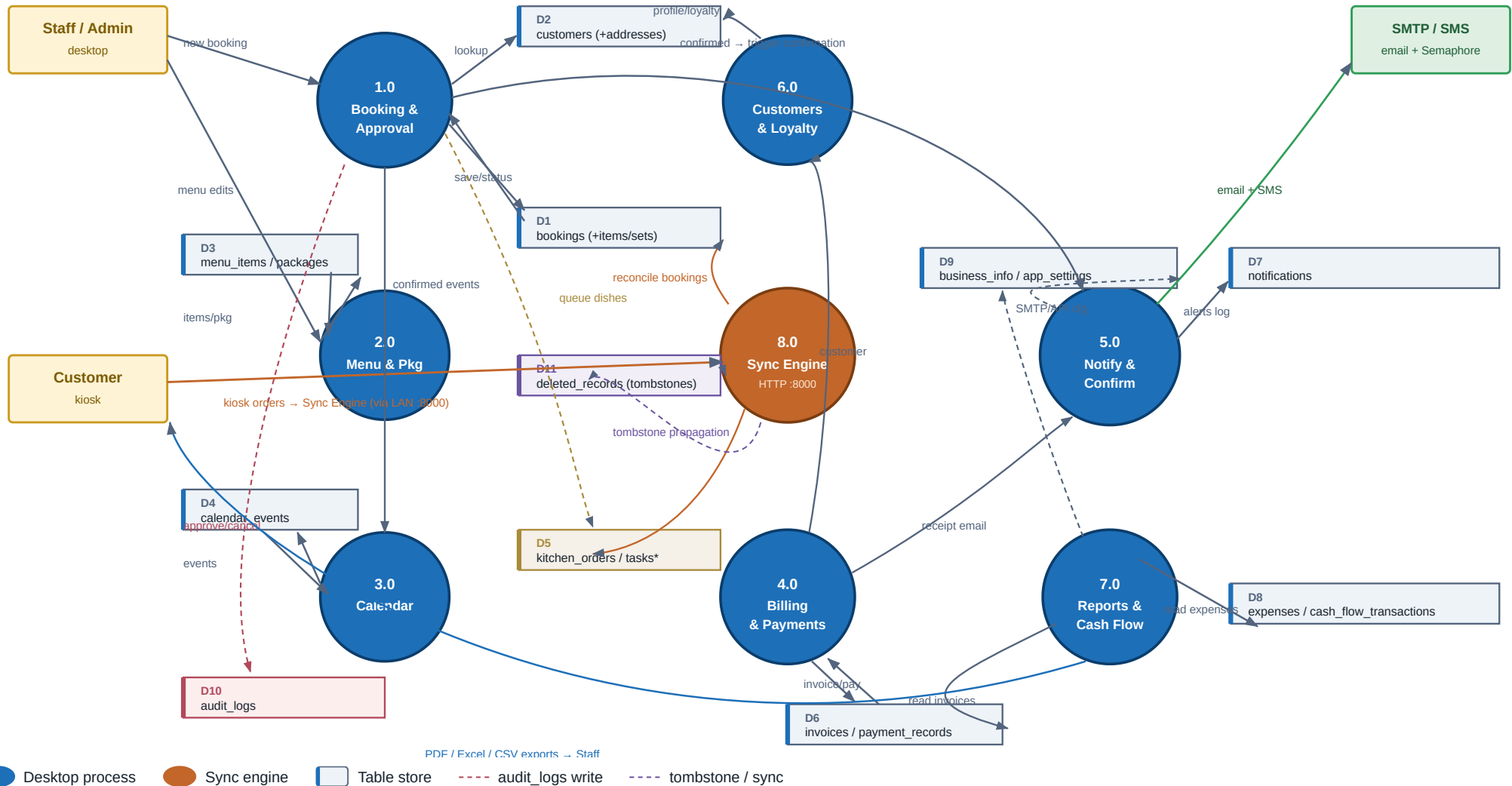


The kiosk is fully functional offline against its in-browser SQLite; it reconciles with the desktop's Central Sync Server over LAN HTTP port 8000 — pushing locally-created bookings/customers and pulling the authoritative menu/package catalog. The desktop GUI is the same process that hosts the sync server (server mode) or proxies to a remote one (client mode). Note: `serve.py` reverse-proxies `/api/*` to `:8000`, and the FastAPI `backend/app.py` keeps a separate `catering_pwa.db` and is a legacy/fallback transport — the live frontend does not route CRUD through it.

Data Flow Diagram — Page 3 · Level 1 (Functional Processes & Data Stores)

Desktop application functional decomposition with the real database tables (utils/sqlite_schema.py)

Processes (circles) · Data stores = actual tables (open rectangles) · dashed red = audit_logs write



* **Kitchen (D5)** and **Inventory** page modules exist in the codebase but are **not wired into the desktop navigation** (not in `_PAGE_MODULES`) — shown amber to flag that. The 39-table schema is grouped here into logical stores. The Sync Engine (8.0) is the same HTTP :8000 server from Page 2; it reconciles kiosk-originated rows into the master tables and propagates deletes via `deleted_records` tombstones.